

#Informe Técnico — Trabajo Práctico Integrador 2026

****Ecosistema:**** Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

****Cátedra:**** Paradigmas de Programación

****Grupo 35****

Integrantes:

- * Romero, Matías
- * Núñez, Axel
- * Romero, Julieta
- * Stoppello, Agustín
- * Torres, Alexis

FASE 1: El Problema de los 3 Focos (Desarrollo del Núcleo)

A continuación, se detalla y clasifica taxonómicamente el catálogo de funciones que componen la lógica central del control urbano, diseñadas bajo las directrices estrictas de inmutabilidad absoluta y ausencia de bucles imperativos

1. transicion

* ****Naturaleza:**** Pura. Dado un color actual y un destino, computa el cambio de manera determinista sin alterar ninguna estructura ni el entorno.

* ****Estrategia de Control:**** Función Predicado / Condicional. Utiliza una estructura `cond` para evaluar la validez lógica del cambio físico de las luces.

* ****Impacto en Memoria:**** No destructiva. Retorna una nueva lista literal con el estado y la acción resultante.

* ****Descripción:**** Modela el cambio de luces. Si la transición no es válida bajo las reglas del negocio, devuelve por defecto el estado actual junto al símbolo `accion-por-defecto`.

2. semaforo-en (Originalmente 'timer')

* ****Naturaleza:**** Pura. Para un timestamp específico y una configuración dada, el color resultante siempre será idéntico y carece de efectos secundarios.

* ****Estrategia de Control:**** Condicional y Composición Funcional.

* ****Impacto en Memoria:**** No destructiva.

* ****Descripción:**** Es la función central del sistema. Aplica el operador matemático `mod` sobre el timestamp usando la duración total del ciclo urbano. Mediante rangos condicionales establecidos por las duraciones dinámicas, decide de forma automática qué foco debe activarse en ese segundo exacto.

3. mostrarCambio

* ****Naturaleza:**** Impura. Rompe el aislamiento funcional al realizar operaciones de Entrada/Salida (E/S) escribiendo directamente en el flujo de la terminal.

* ****Estrategia de Control:**** Secuencial / Efecto Secundario.

* ****Impacto en Memoria:**** No destructiva.

* ****Descripción:**** Actúa como el sistema de auditoría forense primario, capturando el estado de las luces en formato Unix Epoch e imprimiendo la traza legible mediante la directiva `format`.

4. duracionCiclo

- * **Naturaleza:** Pura. Realiza un cálculo matemático estrictamente aritmético.
- * **Estrategia de Control:** Composición Funcional simple.
- * **Impacto en Memoria:** No destructiva.
- * **Descripción:** Determina la extensión temporal completa (en segundos) que le toma al semáforo realizar una rotación entera (Rojo → Amarillo → Verde).

5. recomendacionCiclo

- * **Naturaleza:** Pura.
- * **Estrategia de Control:** Función Predicado / Condicional.
- * **Impacto en Memoria:** No destructiva.
- * **Descripción:** Evalúa la duración calculada del ciclo contra los estándares internacionales de la ingeniería de tráfico (rango óptimo entre 35 y 150 segundos) basados en la psicología del conductor, emitiendo una advertencia o una confirmación de optimización.

6. ciclosPorTiempo

- * **Naturaleza:** Pura.
- * **Estrategia de Control:** Composición Funcional.
- * **Impacto en Memoria:** No destructiva.
- * **Descripción:** Transpone una unidad de tiempo expresada en minutos a segundos para calcular, mediante una división entera, la cantidad exacta de ciclos completos realizables para la planificación logística de mantenimiento de la vía.

7. distribucionHora

- * **Naturaleza:** Pura.
- * **Estrategia de Control:** Composición Funcional.
- * **Impacto en Memoria:** No destructiva.
- * **Descripción:** Calcula y desglosa proporcionalmente el porcentaje de tiempo que cada foco individual permanecerá encendido a lo largo de una hora de operación continua.

FASE 2 — Integración de Quicklisp y cl-json

Introducción

En esta fase se incorporó al sistema de semáforo una librería externa a través del gestor de paquetes Quicklisp, con el objetivo de eliminar los valores de temporización fijos del código fuente y reemplazarlos por una configuración dinámica leída desde un archivo JSON externo.

La librería seleccionada fue `cl-json`, que provee funcionalidades de parseo y serialización del formato JSON dentro del entorno Common Lisp.

Intérprete utilizado: SBCL

Para esta fase se utilizó Steel Bank Common Lisp (SBCL) como intérprete. La elección no es arbitraria: durante el desarrollo se intentó utilizar CLISP a través del plugin SublimeREPL de Sublime Text 3, pero se descartó por las siguientes razones:

****Incompatibilidad con la instalación de Quicklisp:**** Quicklisp se instala de forma independiente para cada implementación de Common Lisp. La instalación realizada fue sobre SBCL. Al intentar cargar ``core.lisp`` desde CLISP, el intérprete no encontraba el archivo `~/quicklisp/setup.lisp`` ya que dicha instalación no le pertenece.

****Madurez y soporte:**** SBCL es la implementación de Common Lisp con mayor actividad de desarrollo activo, mejor soporte de ASDF/Quicklisp y mensajes de compilación más detallados, lo que facilita la detección de errores en tiempo de carga.

Por lo tanto, todos los ejemplos y pruebas de esta fase se ejecutaron en SBCL desde Sublime Text 3, ingresando a él de la siguiente forma:

1. En la barra superior ingresar a ****Tools****, seleccionar ****SublimeREPL****.
2. En el plugin SublimeREPL, buscar ****CommonLisp**** y seleccionar.
3. En CommonLisp seleccionar ****SBCL****.

Esto abrirá una pestaña en Sublime con el intérprete correspondiente. Este intérprete también puede ser utilizado desde PowerShell con el comando:

```
```bash
sbcl --load core.lisp
```

### Instalación de Quicklisp

Quicklisp es un gestor de paquetes para Common Lisp que permite descargar e instalar librerías de forma automatizada desde su repositorio centralizado.

Pasos realizados:

1. Se descargó el archivo instalador [quicklisp.lisp](https://www.quicklisp.org/beta/) desde el sitio oficial: <https://www.quicklisp.org/beta/>
2. Se cargó el instalador en SBCL desde la terminal:

```
Bash
sbcl --load quicklisp.lisp
```

3. Dentro del REPL de SBCL, se ejecutaron los siguientes comandos:  
Lisp  
;; Instala Quicklisp en la carpeta de usuario (~/quicklisp/)  
(quicklisp-quickstart:install)  
;; Agrega la carga automática al archivo de inicio de SBCL (~/sbclrc)  
;; para que Quicklisp esté disponible en cada sesión sin cargarlo manualmente  
(ql:add-to-init-file)

Con esto, Quicklisp quedó instalado en el directorio `~/quicklisp/` y se configuró para cargarse automáticamente cada vez que se inicia SBCL.

### Instalación de cl-json

Con Quicklisp disponible, la instalación de `cl-json` se realizó con un único comando desde el REPL:

```
Lisp
(ql:quickload "cl-json")
```

Para verificar su correcto funcionamiento se ejecutó:

```
Lisp
(json:decode-json-from-string "{\"rojo\": 90}")
;; Devolviendo → ((:ROJO . 90))
```

La librería convierte las claves del JSON en keywords de Common Lisp (símbolos precedidos por `:`), y los pares clave-valor en una lista de asociación (*association list* o *alist*), una estructura nativa del lenguaje.

### Archivo de configuración externa

Se creó el archivo `config.json` en el mismo directorio que `core.lisp`, con los tiempos de temporización:

```
JSON
{
 "rojo": 90,
 "amarillo": 6,
 "verde": 120
}
```

De esta forma, modificar los tiempos del semáforo ya no requiere editar el código fuente, sino únicamente este archivo externo.

### Cambios en el código

Versión preliminar (descartada): uso de variable global

La primera aproximación a la solución implicó crear una variable global `config` que almacenara la configuración leída del archivo JSON, y una función `cargar-config` que la modificara mediante `setf`:

```
Lisp
;; Variable global mutable — DESCARTADO
(defvar config nil)
```

```
;; Función que modifica la variable global — DESCARTADO
(defun cargar-config (ruta)
 (with-open-file (stream ruta :direction :input)
 (setf config (json:decode-json stream))
)
)
```

```
;; Las funciones leían de la variable global implícitamente
(defun duracionCiclo ()
 (+ (obtener-tiempo config :rojo)
 (obtener-tiempo config :amarillo)
 (obtener-tiempo config :verde))
)
```

Esta implementación fue descartada porque viola una restricción explícita de la consigna: queda prohibido el uso de variables globales mutables (`defvar`, `defparameter`) para almacenar estados cambiantes, así como el uso de operadores destructivos (`setq`, `setf`). El estado del sistema debe fluir únicamente a través de los argumentos de las funciones.

Versión final: el config fluye como argumento

La solución correcta elimina por completo la variable global. La función `cargar-config` simplemente retorna el resultado del parseo sin almacenarlo en ningún lugar. Todas las funciones que necesitan los tiempos reciben la configuración como parámetro explícito.

Carga de Quicklisp y `cl-json` al inicio del archivo:  
Lisp

```
(load "~/quicklisp/setup.lisp")
(ql:quickload "cl-json" :silent t)}
```

Esta forma de cargar Quicklisp lanza advertencias normales e inofensivas en el REPL que redefinen funciones internas de la librería y no afectan al funcionamiento del sistema:

Plaintext

```
WARNING: redefining QL-SETUP:QMERGE in DEFUN
WARNING: redefining QL-SETUP:QENOUGH in DEFUN
WARNING: redefining QL-SETUP::IMPLEMENTATION-SIGNATURE in DEFUN
```

## Nuevas funciones agregadas

Lisp

```
=====
;; FUNCION: cargar-config
;; NATURALEZA: Impura
;; ESTRATEGIA: Lectura de archivo externo y parseo JSON
```

:: IMPACTO: No destructiva

=====

```
(defun cargar-config (ruta)
 (with-open-file (stream ruta :direction :input)
 (json:decode-json stream)
)
)
```

**cargar-config** es impura porque realiza entrada/salida (lectura de un archivo del sistema). Sin embargo, no modifica ningún estado externo: simplemente retorna el valor parseado.

Lisp

=====

```
:: FUNCION: obtener-tiempo
:: NATURALEZA: Pura
:: ESTRATEGIA: Búsqueda en lista de asociacion (assoc)
:: IMPACTO: No destructiva
```

=====

```
(defun obtener-tiempo (config color)
 (cdr (assoc color config))
)
```

**obtener-tiempo** es pura: dado el mismo **config** y el mismo **color**, siempre devuelve el mismo resultado de la *alist* mediante **assoc** y **cdr** sin efectos secundarios.

Funciones modificadas para aceptar config como parámetro

Las funciones pasaron de no recibir argumentos de estado a recibir también **config**, recuperando su condición de Puras al ser completamente deterministas:

Función original	Función actualizada
(defun duracionCiclo ())	(defun duracionCiclo (config))
(defun timer (timestamp))	(defun semaforo-en (timestamp config))

(defun distribucionHora ( ))	(defun distribucionHora (config))
(defun ciclosPorTiempo (minutos))	(defun ciclosPorTiempo (minutos config))

## Renombrado de la función timer

Durante las pruebas en SBCL se presentó el siguiente error:

debugger invoked on a SYMBOL-PACKAGE-LOCKED-ERROR:

Lock on package SB-EXT violated when proclaiming TIMER as a function

El nombre **TIMER** está reservado por el paquete interno **SB-EXT** de SBCL. Para evitar violar el lock del paquete, la función fue renombrada a **semaforo-en**, resultando autoexplicativa: (**semaforo-en 91 config**)  $\rightarrow$  **VERDE**.

## Bitácora de Depuración (Bugs Detectados)

1. Bug de Bloqueo de Paquete: Tratamiento del símbolo reservado **TIMER** en SBCL solucionado mediante el renombrado a **semaforo-en**.
2. Bug de Mutación de Estado: Descarte del prototipo inicial con variables globales (**defvar** y **setf**) para pasar a un flujo puramente parametrizado por argumentos.

## FASE 3 — Estudio Comparativo: Scheme

### Introducción a Scheme e Impacto Industrial

Scheme es un dialecto de la familia de lenguajes Lisp, co-creado por Guy L. Steele y Gerald Jay Sussman en los años 70. Se caracteriza por su enfoque en el minimalismo conceptual, una especificación sintáctica sumamente reducida y un diseño elegante.

A nivel industrial, Scheme se utiliza principalmente en:

**Educación y Computación Académica:** Es el lenguaje insignia para la enseñanza de la estructura e interpretación de programas informáticos (como en el célebre libro SICP del MIT).

**Sistemas de Extensión y Scripting:** Su variante *GNU Guile* es el lenguaje oficial de extensión del proyecto GNU, utilizado para configurar entornos del sistema.

**Empresas Destacadas:** Compañías como Naughty Dog han utilizado variantes basadas en Scheme para el scripting de mecánicas y comportamientos de IA en videojuegos (como la saga *Crash Bandicoot*).

### Ejes Comparativos Analíticos

## 1. Espacio de Nombres: Lisp-2 vs. Lisp-1 y la omisión de `funcall`

Common Lisp es clasificado como un Lisp-2, lo que significa que las funciones y las variables residen en espacios de nombres (namespaces) separados. Por ende, para pasar una función como argumento en Common Lisp se debe usar el operador `#'` para obtener el objeto función y, posteriormente, invocarlo mediante la primitiva `funcall`.

Por el contrario, Scheme es un Lisp-1: las funciones y las variables comparten el mismo espacio de nombres idéntico. En Scheme, el identificador de una función es tratado exactamente igual que cualquier otra variable que contiene un valor. Por esta razón técnica, no hace falta utilizar `funcall` ni el operador `#'`. Para ejecutar una función recibida como argumento, basta con colocar el símbolo en la primera posición de una lista evaluable: `(fun_argumento datos)`.

## 2. Optimización de Llamada de Cola (Tail Call Optimization - TCO)

Por especificación oficial del estándar de Scheme, todos los compiladores e intérpretes están obligados a implementar TCO. Esto significa que si una función realiza una llamada recursiva como su última acción absoluta (el resultado de la llamada no requiere cálculos adicionales pendientes en el marco de la pila), el entorno de Scheme no añade un nuevo marco a la pila de ejecución, sino que reutiliza el marco actual.

En nuestra función `semaforo-en` de `solucion.scm`, estructuramos el flujo de datos utilizando la macro de enlace secuencial `let*`. Al evaluar las ligaduras internas de los tiempos dinámicos extraídos de la lista de asociación, la expresión condicional `cond` resuelve el color de forma atómica y directa en sus cláusulas de salida. Si hubiésemos necesitado un ciclo continuo de simulación a través del tiempo, la estructura recursiva se habría diseñado de la siguiente manera para garantizar que el compilador no agote la pila de memoria (Stack Overflow):

Scheme

```
(define (bucle-simulacion-semaforo timestamp config)
 ;; La llamada recursiva ocurre al final sin operaciones pendientes fuera de ella
 (bucle-simulacion-semaforo (+ timestamp 1) config))
```

## ## BITÁCORA DE DEPURACIÓN

**\*\*Bug de Bloqueo de Paquete Interno (Lisp):\*\*** Al intentar nombrar la función reguladora como ``timer``, SBCL interrumpió la carga lanzando un error del tipo ``SYMBOL-PACKAGE-LOCKED-ERROR``. Causa: El símbolo ``timer`` está protegido dentro del paquete interno ``SB-EXT`` de la implementación. Solución: Se renombró la función a ``semaforo-en``, logrando además una semántica más clara para la interacción en el REPL.

**\*\*Bug de Mutación de Estado por Prototipado (Lisp):\*\*** En las primeras iteraciones de la Fase 2, se implementó una aproximación incorrecta declarando un estado global variable con ``(defvar config nil)`` y alterándolo mediante la función destructiva ``setf``. Causa:



Pensamiento arraigado al paradigma imperativo. Solución: Se eliminó la variable global y se reestructuró el sistema para que la configuración dinámica sea puramente devuelta por `cargar-config` y transite exclusivamente como parámetro.

3. **\*\*Bug del Predicado de Igualdad de Símbolos (Migración a Scheme):\*\*** Al transponer la lógica de comparación de estados al archivo `.scm`, las funciones arrojaban respuestas erróneas o caían en la cláusula por defecto. Causa: Intentar utilizar el operador primitivo `eq` de Common Lisp. Solución: Se reemplazó por el predicado nativo correspondiente exigido por el estándar de Scheme: `eq?`.

4. **\*\*Bug de Tipos de Keywords en la Alist (Migración a Scheme):\*\*** La función `assoc` fallaba consistentemente al mapear los tiempos dinámicos trasladados desde el archivo JSON en el entorno de Scheme. Causa: Scheme no posee de forma nativa la sintaxis de *\*Keywords\** precedidos por dos puntos (`:rojo`) tal como lo parsea la librería `cl-json` de Common Lisp. Solución: Se adaptaron los identificadores del JSON dentro de la estructura asociativa convirtiéndolos en símbolos puros y literales del lenguaje mediante el uso de la comilla simple (`'rojo`, `'verde`).

## ## BIBLIOGRAFÍA

\* Steele, G. L., & Sussman, G. J. (1975).

\* Racket Language Documentation — Standards: R5RS & R6RS

(<https://docs.racket-lang.org>).

\*Scheme: An Interpreter for Extended Lambda Calculus

\*. MIT Artificial Intelligence Laboratory.

\* Seibel, P. (2005). *\*Practical Common Lisp\**. Apress. \* Quicklisp Beta Release documentation (<https://www.quicklisp.org>).